

Define la función indice_invertido.

pid → identificador del proceso actual
num_procesos → cantidad total de procesos
lista_documento → lista con todos los documentos
indice → diccionario compartido entre procesos
cerrojo → lock/mutex para evitar conflictos

1

import multiprocessing

2

3

def indice_invertido(pid, num_procesos, lista_documento, indice, cerrojo):

4

for x in range(pid, len(lista_documento), num_procesos):

5

documento = lista_documento[x]

6

nom_archivo = documento["Nombre"]

7

palabras = documento["Palabras"]

8

9

for palabra in palabras:

10

cerrojo.acquire()

11

try:

12

if palabra not in indice:

13

indice[palabra] = []

14

lista_aux = indice[palabra]

15

if nom_archivo not in lista_aux:

16

lista_aux.append(nom_archivo)

17

indice[palabra] = lista_aux

18

finally:

19

cerrojo.release()

20

21

if __name__ == "__main__":

22

cantidad_procesos = 5

23

n_documentos = int(input().strip())

24

25

lista_documentos = []

26

for x in range(n_documentos):

27

nombre_archivo = input().strip()

28

cantidad_terminos = int(input().strip())

29

palabras = input().strip().split()

30

lista_documentos.append({"Nombre" : nombre_archivo, "Palabras" : palabras})

31

32

num_consultas = int(input().strip())

33

consultas = input().strip().split()

34

35

manager = multiprocessing.Manager()

36

indice = manager.dict()

37

cerrojo = multiprocessing.Lock()

38

procesos = []

39

40

for x in range(cantidad_procesos):

41

p = multiprocessing.Process()

42

target = indice_invertido,

43

args = (x, cantidad_procesos, lista_documentos, indice, cerrojo)

44

procesos.append(p)

45

46

for p in procesos:

47

p.start()

48

for p in procesos:

49

p.join()

50

51

for consulta in consultas:

52

if consulta in indice:

53

resultado = " ".join(indice[consulta])

54

print(f'Resultados para "{consulta}": {resultado}')

else:

print(f'Resultados para "{consulta}": No hay resultados.')

Este for divide los documentos entre procesos.3er parametro (num_procesos) es el salto.Esto evita que dos procesos trabajen el mismo documento.

Obtiene un documento específico de la lista

Extrae el nombre del archivo.

Obtiene la lista de palabras del documento.

Recorre todas las palabras del documento.

El proceso toma el lock (cerrojo)

Inicia un bloque protegido.

Verifica si la palabra ya existe en el índice.

Si no existe:crea una lista vacía

Obtiene la lista de documentos asociados a esa palabra.

Verifica que el documento no esté repetido. Esto evita duplicados

Agrega el nombre del documento a la lista.

Actualiza el diccionario compartido.

Este bloque SIEMPRE se ejecuta.

Libera el lock. Permite que otro proceso acceda al diccionario.

Esto asegura que:el código principal solo se ejecute una vez. Es MUY importante en multiprocessing. Sin esto:los procesos podrían crearse infinitamente.

Define que habrá 5 procesos trabajando.

Lee cuántos documentos se ingresarán.

Crea una lista vacía para guardar documentos

Recorre todos los documentos

Lee el nombre del archivo.

Lee la cantidad de palabras.

Lee una línea de texto y la divide en palabras.

Guarda el documento como diccionario.

Lee cuántas consultas se harán.

Lee las palabras a buscar.

Crea un administrador compartido entre procesos.

Crea un diccionario compartido. Todos los procesos pueden modificarlo.

Crea un lock/candado. Sirve para sincronizar procesos.

Lista donde se guardarán los procesos.

Crea todos los procesos.

Crea un proceso nuevo.

Indica qué función ejecutará el proceso.

Argumentos enviados a la función.

Guarda el proceso en la lista.

Inicia todos los procesos. Cada proceso comienza a ejecutar: indice_invertido

El programa espera hasta que TODOS los procesos terminen. Sin esto:podrían hacerse consultas antes de terminar el índice

Recorre cada palabra buscada.

Verifica si existe la palabra.

Une los documentos en un string.

Muestra los resultados encontrados.

Muestra que no hubo coincidencias.

Python

["doc1.txt", "doc2.txt"]

Python

["doc1.txt", "doc2.txt", "doc3.txt"]

Entrada:

Python

python codigo procesos

Resultado:

Python

["python", "codigo", "procesos"]

Python

{

"Nombre": "doc1.txt",

"Palabras": ["python", "codigo"]

}

Python

["doc1", "doc2"]

Python

"doc1 doc2"

Python

Resultados para "python": doc1 doc2